

Xây dựng quan hệ truy hồi và ứng dụng trong thi tin học

Gao Hanrui

Nguồn gốc: Constructing recurrence relations and their applications in informatics competitions

IOI China candidate team papers / 2000 / Gao Hanrui.pdf

Từ khóa. Quan hệ truy hồi; xây dựng; ứng dụng.

Tóm Tắt

Mọi sự vật trên thế giới đều âm thầm biến đổi. Trong vô số biến đổi ấy, nhiều hiện tượng vận động theo quy luật. Quy luật đó thường thể hiện quan hệ giữa nguyên nhân phía trước và kết quả phía sau, vì vậy ta có thể dùng tư tưởng truy hồi để nghiên cứu các biến đổi này. Bài viết tập trung trình bày cách xây dựng quan hệ truy hồi, rồi giới thiệu ngắn gọn các phương pháp giải quan hệ truy hồi. Tiếp đó, bài viết làm rõ quan hệ giữa quan hệ truy hồi và quy hoạch động, so sánh điểm giống và khác giữa quan hệ truy hồi thông thường với quy hoạch động, đồng thời dùng ví dụ để minh họa ứng dụng của quan hệ truy hồi trong thi lập trình. Cuối cùng, bài viết kết luận rằng học tốt quan hệ truy hồi không chỉ nâng cao năng lực toán học mà còn rất hữu ích cho các kỳ thi tin học.

Contents

1	Dẫn nhập	1
2	Định nghĩa quan hệ truy hồi	1
3	Xây dựng quan hệ truy hồi	2
3.1	Năm dạng quan hệ truy hồi điển hình	2
3.1.1	Dãy Fibonacci	2
3.1.2	Bài toán tháp Hanoi	3
3.1.3	Bài toán chia mặt phẳng	3
3.1.4	Số Catalan	4
3.1.5	Số Stirling loại hai	4
3.2	Phương pháp giải quan hệ truy hồi	5
4	Ứng dụng của quan hệ truy hồi	5
5	Kết luận	8
A	Chú thích và chứng minh	9

B Tài liệu tham khảo	10
C Mô tả chương trình	10

1 Dẫn nhập

Trong thế giới biến đổi từng khoảnh khắc, mỗi sự vật đều thay đổi tinh tế theo dòng thời gian. Giữa muôn vàn biến đổi ấy, sự thay đổi của nhiều hiện tượng lại có quy luật; quy luật này thường biểu hiện quan hệ nhân quả. Nói cách khác, kết quả thay đổi của một hiện tượng nào đó liên hệ chặt chẽ với một hoặc một vài kết quả ngay trước nó. Câu nói “nhìn trẻ lên ba biết về sau” cũng nói đến ý này. Ý ấy chính là tư tưởng truy hồi.

Quan hệ truy hồi có vai trò quan trọng trong hầu như mọi nhánh của toán học. Trong Olympic Tin học hiện nay, nơi mọi lời giải đều hướng tới “nhanh hơn, cao hơn, mạnh hơn”, quan hệ truy hồi càng thể hiện sức hấp dẫn riêng nhờ sự gọn gàng và hiệu quả. Khi quan hệ truy hồi giữ vai trò quan trọng như vậy, việc nghiên cứu sâu tính chất và đặc điểm của nó là rất cần thiết. Bài viết này xoay quanh một trong ba vấn đề cơ bản của quan hệ truy hồi: làm thế nào để xây dựng quan hệ truy hồi, đồng thời dùng các bài ví dụ để minh họa ứng dụng của nó trong các kỳ thi tin học hiện nay.

2 Định nghĩa quan hệ truy hồi

Hầu mọi người đều không xa lạ với quan hệ truy hồi, nhưng nếu hỏi chính xác một biểu thức phải thỏa điều kiện gì để được gọi là quan hệ truy hồi, có lẽ mỗi người sẽ có cách nói khác nhau. Để nghiên cứu tốt hơn, trước hết ta cần làm rõ quan hệ truy hồi là gì.

Định nghĩa 1. Cho một dãy số $H_0, H_1, \dots, H_n, \dots$. Nếu tồn tại số nguyên n_0 sao cho khi $n \geq n_0$, ta có thể dùng dấu bằng, hoặc dấu lớn hơn, nhỏ hơn, để liên hệ H_n với một số hạng đứng trước nó $H_i, 0 \leq i < n$, thì biểu thức ấy được gọi là một **quan hệ truy hồi**.

3 Xây dựng quan hệ truy hồi

Trong quan hệ truy hồi có ba vấn đề cơ bản: xây dựng quan hệ truy hồi như thế nào, quan hệ truy hồi đã cho có tính chất gì, và giải quan hệ truy hồi như thế nào. Nếu làm rõ được cả ba mặt này, nhận thức của ta về quan hệ truy hồi sẽ tiến thêm một bước. Do khuôn khổ bài viết, phần này tập trung vào một trong ba vấn đề ấy: xây dựng quan hệ truy hồi.

Mấu chốt của việc xây dựng quan hệ truy hồi là tìm công thức liên hệ số hạng thứ n với một vài số hạng phía trước, cùng với giá trị của các số hạng ban đầu. Đây không phải một khái niệm trừu tượng tách rời bài toán, mà phải đặt trong một bài cụ thể hoặc một lớp bài cụ thể. Dưới đây, ta sẽ thảo luận tương đối cụ thể năm dạng quan hệ truy hồi điển hình.

3.1 Năm dạng quan hệ truy hồi điển hình

3.1.1 Dãy Fibonacci

Trong mọi quan hệ truy hồi, dãy Fibonacci có lẽ là dạng quen thuộc nhất. Ngay trong ngôn ngữ lập trình cơ bản Logo đã có rất nhiều bài thuộc loại này. Khi chuyển sang các ngôn ngữ phức tạp hơn như Basic, Pascal, C, các bài dạng dãy Fibonacci dần rời khỏi sân khấu thi đấu vì lời giải tương đối dễ. Tuy nhiên,

điều đó không có nghĩa dãy Fibonacci không còn giá trị nghiên cứu. Trái lại, một số bài thuộc loại này vẫn có thể gợi cho ta nhiều điều.

Bài toán tiêu biểu của dãy Fibonacci là bài “sinh sản của thỏ”, do nhà toán học Italy Fibonacci nêu ra năm 1202, còn gọi là bài toán Fibonacci.

Bài toán. Có một cặp thỏ đẻ cái. Giả sử sau hai tháng, chúng có thể sinh thêm một cặp thỏ con gồm một đực một cái. Hỏi sau n tháng có tất cả bao nhiêu cặp thỏ?

Lời giải. Gọi F_x là số cặp thỏ sau đủ x tháng, trong đó số thỏ sinh mới trong tháng ấy là N_x , còn số thỏ còn lại từ tháng $x - 1$ là O_x . Khi đó

$$F_x = N_x + O_x.$$

Mặt khác,

$$O_x = F_{x-1}, \quad N_x = O_{x-1} = F_{x-2},$$

vì mọi cặp thỏ ở tháng $x - 2$ đều đã có khả năng sinh sản vào tháng x . Do đó

$$F_x = F_{x-1} + F_{x-2}, \quad F_0 = 0, \quad F_1 = 1.$$

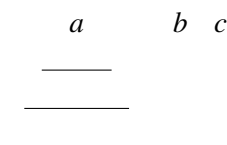
Từ quan hệ truy hồi trên, lần lượt suy ra

$$F_2 = 1, \quad F_3 = 2, \quad F_4 = 3, \quad F_5 = 5, \dots$$

Dãy Fibonacci thường xuất hiện trong các bài đếm tổ hợp đơn giản. Chẳng hạn, bài phủ bàn cờ bằng domino trong các kỳ thi trước đây, Ví dụ 1 ở phần sau, và nhiều bài khác đều có thể giải bằng cách này. Trong phương pháp tối ưu hóa một chiều, dãy Fibonacci cũng thể hiện được tác dụng tốt.

3.1.2 Bài toán tháp Hanoi

Bài toán. Tháp Hanoi gồm n đĩa tròn có kích thước khác nhau và ba cọc a, b, c . Ban đầu, n đĩa được xếp trên cọc a , từ lớn đến nhỏ theo chiều từ dưới lên. Có thể hình dung trạng thái ban đầu như sau:



Cần chuyển toàn bộ n đĩa từ cọc a sang cọc c theo các quy tắc:

1. Mỗi lần chỉ được chuyển một đĩa.
2. Đĩa chỉ được đặt trên một trong ba cọc.
3. Trong quá trình chuyển, không được đặt đĩa lớn lên trên đĩa nhỏ.

Hỏi để chuyển n đĩa từ a sang c , tổng cộng cần bao nhiêu lần chuyển đĩa?

Lời giải. Gọi h_n là số lần chuyển cần thiết để chuyển n đĩa từ a sang c . Rõ ràng khi $n = 1$, chỉ cần chuyển trực tiếp đĩa trên cọc a sang c , nên $h_1 = 1$. Khi $n = 2$, trước hết chuyển đĩa nhỏ từ a sang b , sau đó chuyển đĩa lớn từ a sang c , cuối cùng chuyển đĩa nhỏ từ b sang c , tổng cộng 3 lần, nên $h_2 = 3$.

Tổng quát, khi cọc a có $n \geq 2$ đĩa, ta luôn mượn cọc c để chuyển $n - 1$ đĩa phía trên sang cọc b , rồi chuyển đĩa dưới cùng từ a sang c , sau đó mượn cọc a để chuyển $n - 1$ đĩa từ b sang c . Tổng số lần chuyển là $h_{n-1} + 1 + h_{n-1}$, do đó

$$h_n = 2h_{n-1} + 1, \quad h_1 = 1.$$

3.1.3 Bài toán chia mặt phẳng

Bài toán. Cho n đường cong kín được vẽ trên mặt phẳng. Mọi cặp đường cong kín giao nhau đúng hai điểm, và không có ba đường cong nào cùng đi qua một điểm. Hỏi các đường cong kín ấy chia mặt phẳng thành bao nhiêu miền?

Trong hình gốc, với $n = 1, 2, 3, 4$ có lần lượt 2, 4, 8, 14 miền; quan sát cho thấy các hiệu là 2, 4, 6.

Lời giải. Gọi a_n là số miền mà n đường cong kín chia mặt phẳng thành. Từ các hình đầu tiên ta thấy

$$a_2 - a_1 = 2, \quad a_3 - a_2 = 4, \quad a_4 - a_3 = 6.$$

Những công thức này gợi ý $a_n - a_{n-1} = 2(n-1)$. Tất nhiên, kết luận rút ra từ quan sát vài hình chưa đủ bảo đảm đúng; ta chứng minh như sau.

Giả sử $n-1$ đường cong đã chia mặt phẳng thành a_{n-1} miền. Khi thêm đường cong thứ n , mỗi lần nó cắt một đường cong cũ thì số miền tăng thêm một. Vì đã có $n-1$ đường cong kín, đường cong thứ n cắt mỗi đường cong cũ đúng hai điểm, và không có giao điểm chung của ba đường, số miền tăng thêm là $2(n-1)$. Cộng với a_{n-1} miền đã có, ta được

$$a_n = a_{n-1} + 2(n-1), \quad a_1 = 2.$$

Bài toán chia mặt phẳng là một lớp bài thường xuất hiện trong thi đấu. Vì tính linh hoạt và nhiều biến thể, nó hay làm thí sinh lúng túng. Ví dụ 2 ở phần sau là một dạng chia mặt phẳng khác; độc giả có thể thử tự tìm quan hệ truy hồi trước.

3.1.4 Số Catalan

Số Catalan trước hết được Euler thu được khi tính chính xác số cách chia một đa giác lồi n cạnh thành các tam giác bằng các đường chéo khác nhau. Nó thường xuất hiện trong các bài đếm tổ hợp.

Bài toán. Trong một đa giác lồi n cạnh, dùng các đường chéo không cắt nhau bên trong đa giác để chia đa giác thành các tam giác. Gọi h_n là số cách chia khác nhau; h_n chính là số Catalan tương ứng. Chẳng hạn, ngũ giác có năm cách chia nên $h_5 = 5$. Hãy tìm h_n ứng với một đa giác lồi n cạnh bất kỳ.

Lời giải. Gọi C_n là tổng số cách chia một đa giác lồi n cạnh. Theo yêu cầu của bài toán, mỗi cạnh bất kỳ của đa giác lồi đều là một cạnh của một tam giác; cạnh P_1P_n cũng không ngoại lệ. Vì ba điểm không thẳng hàng xác định một tam giác, chỉ cần chọn một điểm P_k trong các điểm $P_2, P_3, \dots, P_{n-1}$, với $1 < k < n$, để cùng P_1, P_n tạo thành ba đỉnh của một tam giác, ta đã chia đa giác n cạnh thành ba phần không giao nhau.

Trong ba phần ấy, tam giác $P_1P_kP_n$ là một phần cố định; phần còn lại thứ nhất là một đa giác lồi k cạnh, phần còn lại thứ hai là một đa giác lồi $n-k+1$ cạnh. Số cách chia của hai phần này lần lượt là C_k và C_{n-k+1} . Vì vậy, số cách chia của đa giác n cạnh có chứa tam giác $P_1P_kP_n$ là C_kC_{n-k+1} . Điểm P_k có thể là bất kỳ điểm nào trong $P_2, \dots, P_{n-1}$, nên theo nguyên lý cộng,

$$C_n = \sum_{i=2}^{n-1} C_i C_{n-i+1}.$$

Để tiện tính toán, quy ước điều kiện biên là $C_2 = 1$.

Số Catalan là một quan hệ truy hồi tương đối phức tạp. Đặc biệt trong thi đấu, thí sinh khó xây dựng được quan hệ đúng trong thời gian ngắn. Dĩ nhiên, các bài dạng Catalan cũng có thể làm bằng tìm kiếm, nhưng so với phương pháp dựa trên quan hệ truy hồi, tìm kiếm không chỉ kém hiệu quả mà độ phức tạp khi lập trình cũng tăng mạnh.

3.1.5 Số Stirling loại hai

Trong năm dạng quan hệ truy hồi điển hình, số Stirling loại hai có lẽ ít quen thuộc nhất. Vì vậy, trước hết cần giải thích số Stirling loại hai là gì.

Định nghĩa 2. Đặt n quả bóng phân biệt vào m hộp giống nhau, yêu cầu không hộp nào rỗng. Số phương án khác nhau được ký hiệu là $S_2(n, m)$, gọi là số Stirling loại hai.

Dưới đây, ta dựa vào định nghĩa này để suy ra quan hệ truy hồi có hai tham số của số Stirling loại hai.

Lời giải. Giả sử có n quả bóng khác nhau, ký hiệu lần lượt là $b_1, b_2, \dots, b_n$. Lấy riêng quả bóng b_n . Có hai cách xét vị trí của b_n :

1. b_n một mình chiếm một hộp. Khi đó các bóng còn lại chỉ có thể đặt vào $m - 1$ hộp, số phương án là $S_2(n - 1, m - 1)$.
2. b_n ở chung hộp với bóng khác. Khi đó trước hết đặt $b_1, b_2, \dots, b_{n-1}$ vào m hộp, rồi đặt b_n vào một trong m hộp ấy, số phương án là $mS_2(n - 1, m)$.

Tổng hợp hai trường hợp, ta được định lý về số Stirling loại hai:

$$S_2(n, m) = mS_2(n - 1, m) + S_2(n - 1, m - 1), \quad n > 1, m \geq 1.$$

Điều kiện biên suy ra từ định nghĩa:

$$S_2(n, 0) = 0, \quad S_2(n, 1) = 1, \quad S_2(n, n) = 1, \quad S_2(n, k) = 0 \ (k > n).$$

Số Stirling loại hai ít xuất hiện trong thi đấu hơn, nhưng vẫn có một số bài tương tự, thậm chí phức tạp hơn. Độc giả có thể thử tự xây dựng quan hệ truy hồi của chúng.

Tiểu kết. Qua quá trình xây dựng năm quan hệ truy hồi điển hình trên, có thể thấy khi gặp bài thuộc dạng truy hồi, ta phải phân tích theo tình huống cụ thể, tìm mối liên hệ giữa một trạng thái và các trạng thái trước đó, rồi xây dựng quan hệ tương ứng.

3.2 Phương pháp giải quan hệ truy hồi

Phương pháp đơn giản và dễ dùng nhất để giải quan hệ truy hồi chính là tính truy hồi. Đây là một phương pháp trong thuật toán lặp, dùng một số bước tính đơn giản có thể lặp lại để mô tả một bài toán toán học phức tạp, nhờ đó thuận tiện cho máy tính xử lý. Tư tưởng của nó hoàn toàn nhất quán với quan hệ truy hồi: bắt đầu từ điều kiện biên rồi lần lượt tính về sau. Trong trường hợp thông thường, cách này có hiệu suất cao và lập trình rất thuận tiện.

Tuy vậy, thông thường ta chỉ cần số hạng thứ n của quan hệ truy hồi, còn thông tin về các số hạng giữa điều kiện biên và số hạng thứ n thì không cần. Nếu dùng cách tính truy hồi, mọi số hạng trước n đều phải được tính ra, cuối cùng mới thu được giá trị cần tìm. Đây là điểm không tránh được của tính truy hồi, và ở mức độ nhất định sẽ ảnh hưởng đến hiệu suất chương trình. Dĩ nhiên, trong đa số tình huống, phương pháp truy hồi vẫn khả thi; trong thi đấu, chương trình dùng cách này thường có thể chạy trong giới hạn thời gian.

Các phương pháp tốt hơn còn có hàm sinh, quy nạp lặp, v.v.; ở đây không bàn chi tiết. Chẳng hạn, cũng với dãy Fibonacci, cách truy hồi phải bắt đầu từ F_1, F_2 rồi lần lượt tính $F_3, F_4, \dots, F_n$. Còn dùng

hàm sinh, ta có thể thu được công thức

$$F_n = \frac{\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n}{\sqrt{5}},$$

từ đó trực tiếp tính bất kỳ số hạng nào cần dùng.

4 Ứng dụng của quan hệ truy hồi

Quan hệ truy hồi có phạm vi ứng dụng rất rộng. Một ứng dụng rất quan trọng là tam giác Yang Hui, còn gọi là tam giác Pascal. Tam giác này được vẽ theo công thức tổ hợp

$$C_n^r = C_{n-1}^r + C_{n-1}^{r-1}.$$

Rõ ràng, cả công thức tổ hợp lẫn tam giác Pascal đều thuộc phạm vi quan hệ truy hồi. Một phần đầu của tam giác là

$$\begin{array}{ccccccc} & & & & 1 & & & & \\ & & & & & 1 & & 1 & \\ & & & 1 & & 2 & & 1 & \\ & & 1 & & 3 & & 3 & & 1 \\ 1 & & 4 & & 6 & & 4 & & 1 \end{array}$$

Trong các kỳ thi tin học hiện nay, ứng dụng của quan hệ truy hồi cũng ngày càng rộng. Dưới đây ta xem xét một vài bài thi những năm gần đây.

Ví dụ 1. Kỳ thi vòng hai thành phố Bengbu năm 1998, bài 1. Có một con ong đã được huấn luyện, chỉ có thể bò sang ô tổ ong kề bên ở phía phải, không thể bò ngược lại. Hãy tính số đường đi có thể để ong bò từ ô a đến ô b .

Trong hình gốc, các ô tổ ong được đánh số theo hai hàng so le:

$$\begin{array}{cccccc} 3 & 5 & 7 & 9 & 11 & 13 \\ 4 & 6 & 8 & 10 & 12 & 14 \end{array}$$

Lời giải. Đây là một bài rất điển hình thuộc dạng dãy Fibonacci, và quan hệ truy hồi rất rõ ràng. Do hạn chế “ong chỉ có thể bò sang ô kề bên ở phía phải, không thể bò ngược lại”, mọi đường đi đến điểm b chỉ có thể đến từ điểm $b-1$ hoặc $b-2$. Vì vậy,

$$f_n = f_{n-1} + f_{n-2} \quad (a+2 \leq n \leq b),$$

với điều kiện biên

$$f_a = 1, \quad f_{a+1} = 1.$$

Chương trình tương ứng là Pro_1.Pas trong phần mô tả chương trình.

Ví dụ 2. Kỳ thi vòng hai thành phố Hefei năm 1998, bài 2. Trong cùng một mặt phẳng có n đường thẳng, $n \leq 500$. Biết có p đường thẳng, $p \geq 2$, cùng đi qua một điểm. Hỏi n đường thẳng này nhiều nhất có thể chia mặt phẳng thành bao nhiêu miền khác nhau?

Lời giải. Bài này rất giống bài chia mặt phẳng ở phần đầu, khác ở chỗ đường là thẳng hay cong, và có tồn tại các đường đồng quy hay không. Vì tính đặc biệt của các đường đồng quy, ta quyết định trước hết xét p đường thẳng cùng đi qua một điểm, rồi xét $n - p$ đường thẳng còn lại.

Trước hết, có thể trực tiếp tính được p đường thẳng cùng đi qua một điểm chia mặt phẳng thành $2p$ miền. Sau đó, trên cơ sở mặt phẳng đã có $k \geq p$ đường thẳng, thêm một đường thẳng nữa. Đường mới nhiều nhất có thể cắt k đường cũ; mỗi giao điểm làm tăng thêm một miền, và sau giao điểm cuối cùng, vì đường thẳng kéo dài vô hạn, còn tăng thêm một miền nữa. Do đó

$$f_m = f_{m-1} + m \quad (m > p),$$

với điều kiện biên đã tính là $f_p = 2p$. Tuy bài này nhìn qua có hai tham số, khi giải thực tế sẽ thấy nó vẫn thuộc quan hệ truy hồi một tham số. Chương trình tương ứng là Pro_2.Pas.

Hai ví dụ trên có quan hệ truy hồi khá rõ. Nói chúng là bài thuộc loại quan hệ truy hồi chắc hẳn không ai nghi ngờ. Bây giờ ta xét một bài khác.

Ví dụ 3. Kỳ tuyển chọn đội tuyển tỉnh Jiangsu năm 1999, bài 2. Trong một bảng ô vuông $n \times m$, với m lẻ, có đặt $n \times m$ số. Phía dưới ô giữa của bảng có một người. Người này có thể đi theo năm hướng tiến lên, nhưng không được ra ngoài bảng. Mỗi khi đi qua một ô, người đó phải lấy số trong ô ấy. Hãy tìm một đường đi từ đáy lên đỉnh sao cho tổng các số lấy được là lớn nhất, và in ra tổng lớn nhất.

Năm hướng có thể hiểu là từ hàng trước đến hàng sau với độ lệch cột $-2, -1, 0, 1, 2$.

Lời giải. Về bản chất, bài này tương tự Ví dụ 1: từ những điểm có thể đến được từ một điểm, ta tính ngược lại mọi điểm có thể đi đến một điểm, rồi khai thác quan hệ giữa chúng. Dùng tọa độ (x, y) xác định duy nhất một điểm, trong đó (m, n) là góc trên bên phải của hình, còn điểm xuất phát của người là $([m/2], 0)$. Do bị hạn chế bởi các hướng đi, những điểm có thể đến trực tiếp (x, y) chỉ là

$$(x+2, y-1), (x+1, y-1), (x, y-1), (x-1, y-1), (x-2, y-1).$$

Đường đi đến (x, y) có tổng lớn nhất tất nhiên phải sinh ra từ các đường đi tốt nhất đến năm điểm trên; vì yêu cầu phương án tối ưu, ta chọn đường có tổng lớn nhất. Quan hệ là

$$F_{x,y} = \max\{F_{x+2,y-1}, F_{x+1,y-1}, F_{x,y-1}, F_{x-1,y-1}, F_{x-2,y-1}\} + \text{Num}_{x,y},$$

trong đó $\text{Num}_{x,y}$ là số trên điểm (x, y) . Điều kiện biên là

$$F_{[m/2],0} = 0, \quad F_{x,0} = -\infty \quad (1 \leq x \leq m, x \neq [m/2]).$$

Chương trình tương ứng là Pro_3.Pas.

Nhìn bài trên, chắc chắn có người nói rằng đây không phải ứng dụng của quan hệ truy hồi mà là quy hoạch động; công thức trên không phải quan hệ truy hồi mà là phương trình chuyển trạng thái. Vì sao? Vì trong công thức có phép lấy giá trị lớn nhất $\max$, nên nó thuộc quy hoạch động. Có đúng vậy không?

Trong định nghĩa quan hệ truy hồi chỉ yêu cầu “dùng dấu bằng, hoặc dấu lớn hơn, nhỏ hơn, để liên hệ $H(n)$ với một số hạng đứng trước nó $H(i)$, $0 \leq i < n$ ”. Ý nghĩa của “liên hệ” rất rộng, đương nhiên cũng bao gồm việc dùng toán tử lấy lớn nhất hoặc nhỏ nhất để liên hệ. Vì vậy, ta cho rằng bài trên vẫn có thể thuộc quan hệ truy hồi; đồng thời, nó cũng thuộc quy hoạch động.

Vậy quan hệ giữa quan hệ truy hồi và quy hoạch động rốt cuộc là gì? Có thể biểu diễn bằng biểu đồ Venn: nếu $A = \{\text{quan hệ truy hồi}\}$ và $B = \{\text{quy hoạch động}\}$, thì $B \subset A$. Thông thường, khi nói quan hệ truy hồi, người ta hiểu là quan hệ truy hồi thông thường, nên mới cho rằng quy hoạch động và quan hệ truy hồi là hai cá thể độc lập. Bảng sau so sánh điểm giống và khác giữa quan hệ truy hồi thông thường và quy hoạch động.

Mục so sánh	Quan hệ truy hồi thông thường	Quy hoạch động
Hậu hiệu	Không có	Không có
Điều kiện biên	Thường rất rõ ràng	Thường khá ẩn, dễ bị bỏ sót
Dạng tổng quát	$H_n = a_1H_{n-1} + a_2H_{n-2} + \dots + a_rH_{n-r}$; tính toán học mạnh hơn	$H_n = \max\{\dots\} + a$; tính toán học tương đối yếu hơn
Giai đoạn	Thường không chia giai đoạn	Thường có giai đoạn khá rõ

Dù giữa quan hệ truy hồi thông thường và quy hoạch động có nhiều khác biệt, trong thực tế người ta vẫn thường lẫn lộn chúng, và hay nhầm quan hệ truy hồi thông thường thành quy hoạch động. Từ thập niên 1950–1960, quy hoạch động được phát hiện và từng tạo một làn sóng lớn trong thi tin học; cho đến hôm nay, nó vẫn là nội dung trọng tâm. Nhưng quan hệ truy hồi có lịch sử lâu dài. Dù quan hệ truy hồi thông thường hiện không nóng như quy hoạch động, ảnh hưởng của nó vẫn không thể xem nhẹ.

Trong kỳ tuyển chọn IOI 1999 từng có bài “Thống kê 01”. Nhiều người chỉ dùng công thức

$$C_n^r = \frac{n!}{r!(n-r)!}$$

để tính số tổ hợp, rồi lấy tổng các số tổ hợp nhằm tìm số lượng số loại A, dẫn đến hiệu suất chương trình không cao. Thực ra đây là một bài rất điển hình thuộc loại quan hệ truy hồi. Vì thí sinh thường ngày xem nhẹ quan hệ truy hồi thông thường, họ dễ thất bại trong phòng thi.

Còn một lớp bài trò chơi cũng dùng quan hệ truy hồi. Ta xét một ví dụ.

Ví dụ 4. Tạp chí *Student Computer World*, năm 1995. Viết một chương trình để máy tính và người chơi trò lấy bài, cố gắng để máy tính thắng, đồng thời hiển thị toàn bộ quá trình chơi. Trò chơi như sau: có n lá bài, $3 < n < 10000$. Hai người chơi, một bên là máy tính và một bên là người, luân phiên lấy bài; máy tính lấy trước. Ai lấy lá cuối cùng thì thắng. Quy tắc lấy là: người đầu tiên lấy từ 1 đến $n-1$ lá, tức lấy tùy ý nhưng không được lấy hết; sau đó, mỗi người có thể lấy từ 1 lá trở lên, nhưng không được lấy quá gấp đôi số lá đối phương vừa lấy. Nếu đối phương vừa lấy b lá, người tiếp theo có thể lấy từ 1 đến $2b$ lá.

Lời giải. Bài này nhìn qua là một bài quy hoạch động rất rõ ràng. Chia giai đoạn theo số lá còn lại; trạng thái $F_{p,k}$ biểu diễn khi còn p lá và người đi trước nhiều nhất được lấy k lá, vị trí ấy là thua chắc hay thắng chắc. Phương trình chuyển trạng thái là

$$F_{p,k} = (p \leq k) \text{ or } F_{p,k-1} \text{ or not } F_{p-k,2k}, \quad 1 \leq p \leq n, \quad 1 \leq k < n.$$

Sau đó, ta có thể dựa vào các trạng thái đã tính để thiết kế một cách lấy bài khiến máy tính chắc thắng. Dĩ nhiên, nếu số lá ban đầu là trạng thái thua chắc thì máy tính chỉ có thể nhận thua.

Khi số lá không lớn, phương pháp này có hiệu suất khá cao. Nhưng một khi số lá rất lớn, giả sử n đạt cực hạn, không gian cần dùng có bậc $O(10^5 \cdot 10^5)$, tất yếu sẽ thiếu bộ nhớ; phương pháp này không còn khả thi.

Ta thử vẽ các trạng thái có số lá nhỏ thành bảng để quan sát quy luật. Dấu $\surd$ biểu diễn True, dấu $\times$ biểu diễn False.

$n \setminus k$	1	2	3	4	5	6	7
2	$\times$						
3	$\times$	$\times$					
4	$\surd$	$\surd$	$\surd$				
5	$\times$	$\times$	$\times$	$\times$			
6	$\surd$	$\surd$	$\surd$	$\surd$	$\surd$		
7	$\times$	$\surd$	$\surd$	$\surd$	$\surd$	$\surd$	
8	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$

Từ bảng này có thể thấy rõ: nếu số lá còn lại là 2, 3, 5, 8, thì bất kể người đi trước lấy bao nhiêu lá, giả sử không thể lấy hết một lần, người đó chắc chắn sẽ thua, trừ khi đối phương không muốn thắng. Những số như 2, 3, 5, 8 được gọi là **số lá thua chắc**. Khi số lá ban đầu không phải số lá thua chắc, ta cần thiết kế một phương án sao cho mỗi lần máy tính lấy x lá, số lá còn lại lớn hơn $2x$ và là số lá thua chắc.

Vậy rốt cuộc những số nào là số lá thua chắc? Từ các số 2, 3, 5, 8 trong bảng, ta đoán rằng các số Fibonacci bắt đầu từ 2 đều là số lá thua chắc, và có thể chứng minh bằng toán học. Nhờ đó ta có thể thiết kế phương án dựa trên các số thua chắc đã tìm được. Nếu muốn sau mỗi lần máy tính lấy x lá, số lá còn lại vừa là số thua chắc vừa lớn hơn $2x$, điều này có vẻ không phải lúc nào cũng làm được. Chẳng hạn ban đầu có 20 lá; nếu lấy một lần 7 lá để còn 13 lá, đối thủ có thể lấy hết 13 lá trong một lần. Vì vậy, ta chỉ cần thiết kế thêm một phương án cho 7 lá, bảo đảm máy tính lấy được lá thứ 7, và số lá máy tính lấy lần cuối nhỏ hơn $13/2$. Bước này chỉ cần lồng thêm một lần dãy Fibonacci là làm được. Chương trình tương ứng là Pro_4.Pas.

Tiểu kết. Từ các ví dụ trên có thể thấy, dùng quan hệ truy hồi thông thường đôi khi đơn giản hơn quy hoạch động. Khi việc cài đặt quy hoạch động khá khó, quan hệ truy hồi thông thường có thể đóng vai trò quan trọng; tác dụng ấy thường thể hiện ở hai mặt: giải trực tiếp hoặc đơn giản hóa quá trình quy hoạch động.

5 Kết luận

Qua phần trình bày cụ thể ở trên về cách xây dựng quan hệ truy hồi và ứng dụng của nó trong thi tin học, có thể thấy quan hệ truy hồi không phải một khái niệm trừu tượng. Nó cụ thể, cần được xét theo từng bài toán cụ thể. Vì vậy, ta không thể tìm ra một phương pháp chung để xây dựng mọi quan hệ truy hồi, mà chỉ có thể phân tích dựa trên các điều kiện cụ thể của bài cần giải.

Dù việc xây dựng quan hệ truy hồi không có một khuôn mẫu cố định, xét tổng thể, ta đều cần trước hết tìm ra các điều kiện quan trọng của bài toán; trên cơ sở đó phân tích quan hệ giữa một số hạng và một vài số hạng phía trước, rồi tìm điều kiện biên. Ứng dụng của quan hệ truy hồi trong thi đấu rất rộng; nó bao gồm cả quy hoạch động, nội dung hầu như kỳ thi nào cũng kiểm tra. Vì vậy, học tốt cách xây dựng quan hệ truy hồi rất có ích, dù là để nâng cao năng lực toán học hay để chuẩn bị cho các kỳ thi sau này. Vì quy hoạch động quen thuộc hơn với mọi người, bài viết này tập trung vào quan hệ truy hồi thông thường, với hy vọng đem lại một số gợi ý cho thí sinh.

A Chú thích và chứng minh

1. **Phủ bằng domino.** Bàn cờ $2 \times n$ được phủ kín bằng các domino 1×2 . Hỏi số cách phủ khác nhau C_n .
2. **Phương pháp tối ưu hóa một chiều.** Giả sử hàm $y = f(x)$ trên khoảng (a, b) có một điểm cực trị đơn đỉnh, giả sử là cực đại. “Đơn đỉnh” nghĩa là chỉ có một điểm cực trị ξ , và khi x lệch khỏi ξ càng xa thì độ lệch $|f(x) - f(\xi)|$ càng lớn. Yêu cầu dùng một số lần thử nghiệm để tìm ξ chính xác đến một mức nhất định. Các phương pháp thử nghiệm tốt hơn gồm phương pháp 0.618 và phương pháp Fibonacci.
3. Chứng minh công thức tổ hợp

$$C_n^r = C_{n-1}^r + C_{n-1}^{r-1}.$$

Ta có

$$C_{n-1}^r = \frac{(n-1)!}{r!(n-r-1)!}, \quad C_{n-1}^{r-1} = \frac{(n-1)!}{(r-1)!(n-r)!}.$$

Do đó

$$\begin{aligned} C_{n-1}^r + C_{n-1}^{r-1} &= \frac{(n-1)!(n-r) + (n-1)!r}{r!(n-r)!} \\ &= \frac{n!}{r!(n-r)!} = C_n^r. \end{aligned}$$

4. **Dãy 01.** Gần đây, các chuyên gia IOI đang nghiên cứu các số nhị phân. Một bài toán thống kê trong nghiên cứu này làm họ rất đau đầu. Với một số tự nhiên n , có thể đổi nó thành biểu diễn nhị phân $a_k a_{k-1} \dots a_1 a_0$, trong đó

$$n = a_k 2^k + a_{k-1} 2^{k-1} + \dots + a_1 2^1 + a_0,$$

và $a_i = 0$ hoặc 1 với $0 \leq i < k$, $a_k = 1$. Ví dụ $10 = (1010)_2$, $5 = (101)_2$. Ta thống kê $a_0, \dots, a_k$: nếu trong $k+1$ chữ số ấy số lượng 0 nhiều hơn số lượng 1, thì gọi đó là số loại A. Nhiệm vụ là với một m cho trước, tính số lượng số loại A trong đoạn 1 đến m .

5. Cách giải bài “Thống kê 01” có thể tham khảo báo cáo lời giải của Xu Jing.
6. Chứng minh bằng quy nạp rằng các số Fibonacci bắt đầu từ 2 là số lá thua chắc trong trò chơi lấy bài.

Trước hết, từ bảng trạng thái thấy được các số Fibonacci 2, 3, 5 là số lá thua chắc, và mọi số khác trong khoảng $(1, 5)$ là số lá thắng chắc.

Giả sử các số Fibonacci $F_1, F_2, \dots, F_i, F_{i+1}$ thỏa tính chất: trong khoảng $(F_1, F_{i+1}]$, chỉ các số Fibonacci mới là số lá thua chắc, còn các số khác đều là số lá thắng chắc. Ta chứng minh trong khoảng $(F_1, F_{i+2}]$, tính chất trên cũng đúng.

Xét trường hợp còn F_{i+2} lá. Giả sử lần này máy tính lấy x lá, khi đó còn

$$p = F_{i+2} - x$$

lá.

Nếu $x \geq F_i$, thì $p \leq F_{i+1}$. Vì $F_{i+1} = F_i + F_{i-1}$ và $F_{i-1} \leq F_i$, nên $F_{i+1} \leq 2F_i$, do đó $p \leq 2x$. Người chơi có thể lấy hết p lá còn lại trong một lần, nên máy tính chắc chắn thua.

Nếu $1 \leq x < F_i$, thì $p > F_{i+1}$. Vì $F_{i+2} - F_i = F_{i+1}$ và F_{i+1} là số Fibonacci, máy tính không thể thông qua một phương án lấy bài nào đó để sau một lần lấy ít hơn $F_{i+1}/2$ lá mà còn lại F_{i+1} lá. Do đó khi còn F_{i+1} lá, tất yếu đến lượt máy tính lấy, và lúc ấy máy tính không thể lấy hết một lần. Vì F_{i+1} là số Fibonacci, máy tính chắc chắn thua.

Mặt khác, với $p \in (F_{i+1}, F_{i+2})$, khi còn p lá và đến lượt người chơi, người chơi chắc chắn thắng, nên p là số lá thắng chắc. Kết hợp cơ sở quy nạp và bước quy nạp, kết luận được chứng minh.

Các chứng minh trong các mục 3 và 6 là chứng minh của tác giả; nếu có sai sót, mong được chỉ ra.

B Tài liệu tham khảo

1. Yang Zhensheng, biên soạn; Wang Shuhe, hiệu đính. *Combinatorics and its algorithms*. University of Science and Technology of China Press, 1997.
2. Sun Shuling, Xu Yinlong. *Introduction to combinatorics*. University of Science and Technology of China Press, 1999.
3. Lu Kaicheng. *Combinatorics*. Tsinghua University Press, 1991.

4. Wu Wenhua, Wang Jiande. *Guide to youth international and national informatics olympiad: combinatorial algorithms and programming*. Tsinghua University Press, 1997.
5. Wu Wenhua, chủ biên. *Informatics Olympiad*, advanced volume. Peking University Press, 1992.
6. Wu Wenhua, chủ biên. *Informatics Olympiad*, intermediate volume. Peking University Press, 1992.

C Mô tả chương trình

Pro_1.Pas

```

program Pro_1; {Bai toan to ong, khoang toi da giua dich va xuat phat la 40000}
type
  Tarr=array[1..10000] of byte;
  Tnum=array[1..4] of ^Tarr;
  Tlen=array[1..4] of word;
var
  start,finish:longint;
  {Diem xuat phat va diem dich}
  num:Tnum;
  {num[1]..num[3] luu so duong di den ba o to ong lien tiep;
   num[4] la bien tam de hoan doi}
  len:Tlen;
  {len[i] bieu dien do dai cua num[i]}

procedure DataInit; {Khoi tao du lieu}
var
  i:integer;
begin
  for i:=1 to 3 do
    begin
      new(num[i]);
      fillchar(num[i]^,sizeof(num[i]^),0);
      num[i]^[1]:=1; len[i]:=1;
    end;
end;

procedure Add; {Cong do chinh xac cao: num[3]=num[1]+num[2]}
var
  i,carry:word; {carry la chu so nho}
begin
  carry:=0;
  for i:=1 to len[2] do
    begin
      carry:=carry+num[2]^[i]+num[1]^[i];
      num[3]^[i]:=carry mod 10;
      carry:=carry div 10;
    end;
  len[3]:=i;

```

```

    if carry>0 then
    begin
        inc(len[3]);
        num[3]^[i+1]:=carry;
    end;
end;

```

```

procedure Work; {Tinh tong so duong di tu diem xuat phat den diem dich}
var
    i:longint;
begin
    num[1]^[1]:=1; num[2]^[1]:=1;
    for i:=start+2 to finish do
    begin
        Add; {Cong do chinh xac cao: num[3]=num[1]+num[2]}
        num[4]:=num[1];
        num[1]:=num[2];
        num[2]:=num[3];
        num[3]:=num[4];
        len[4]:=len[1];
        move(len[2],len[1],6);
    end;
end;

```

```

procedure Out; {Xuat ket qua}
var
    i:integer;
begin
    for i:=len[2] downto 1 do
        write(num[2]^[i]);
    writeln;
end;

begin
    DataInit; {Khoi tao du lieu}
    write('Input: ');
    readln(start,finish);
    Work; {Tinh tong so duong di tu diem xuat phat den diem dich}
    Out; {Xuat ket qua}
end.

```

Pro_2.Pas

```

program Pro_2; {Bai toan chia mat phang thu hai}
var
    n,p,total,i:longint; {total la tong so mien}
begin
    write('n='); readln(n);

```

```

write('p='); readln(p);
total:=p*2;
for i:=p+1 to n do inc(total,i);
{Cung co the dung total:=p*2+(p+n+1)*(n-p) div 2}
writeln('Total Area:',total);
end.

```

Pro_3.Pas

```

program Pro_3; {Lay so}
const
  Infns='Pro_3.in';
  Sign=-10000000; {Dau hieu cho diem khong the den}
type
  Tarr=array[0..100,1..101] of longint;
var
  num:^Tarr; {Luu so tren moi diem}
  sum:Tarr; {Luu tong lon nhat co the dat den moi diem}
  n,      {Chieu cao cua bang}
  m:integer; {Chieu rong cua bang}

procedure ReadIn; {Doc du lieu va khoi tao}
var
  ni,mi:integer;
begin
  assign(Input,Infns);
  reset(Input);
  readln(n,m);
  new(num);
  for ni:=n downto 1 do
    for mi:=1 to m do
      begin
        read(num^[ni,mi]);
      end;
  for mi:=1 to m do
    sum[0,mi]:=Sign; {Dat dau hieu cho diem khong the den}
  sum[0,m div 2+1]:=0; {Diem xuat phat cua nguoi}
  close(Input);
end;

procedure Work; {Tim tong lon nhat cua duong di den moi diem}
var
  ni,mi,dir:integer;
  max:longint;
begin
  for ni:=1 to n do
    for mi:=1 to m do
      begin

```

```

        max:=sign;
        for dir:=-2 to 2 do
            if (mi+dir>0) and (mi+dir<=m) and (sum[ni-1,mi+dir]>max)
                then max:=sum[ni-1,mi+dir];
            if max>sign then sum[ni,mi]:=max+num^[ni,mi]
                else sum[ni,mi]:=max;
        end;
    end;
end;

```

```

procedure FindMax; {Tim tong lon nhat cua mot duong di}

```

```

var
    mi:integer;
    max:longint;
begin
    max:=sign;
    for mi:=1 to m do
        if sum[n,mi]>max then max:=sum[n,mi];
    writeln('Max=',max)
end;

```

```

begin
    ReadIn; {Doc du lieu va khoi tao}
    Work; {Tim tong lon nhat cua duong di den moi diem}
    FindMax; {Tim tong lon nhat cua mot duong di}
end.

```

Pro_4.Pas

```

program Pro_4; {Bai toan tro choi}

```

```

var
    Youget, {Nguoi choi lay bao nhieu la}
    total:integer; {Hien con bao nhieu la}

```

```

procedure GetYourAnswer(Iget:integer); {Doc so la nguoi choi lay lan nay}

```

```

begin
    repeat
        write('You: '); readln(Youget);
    until (Youget>0) and (Youget<=Iget*2);
    dec(total,Youget); {Tong so la giam tuong ung}
    writeln('Remain: ',total);
end;

```

```

procedure Work(n:integer); {Thiet ke cach lay de may tinh lay duoc la thu n}

```

```

var
    a1,a2,a3:integer;
begin
    if Youget*2>=n then {Co the lay het mot lan hay khong}
        begin

```

```

        writeln('I: ',n); dec(total,n); writeln('Remain: ',total); exit;
    end;
a1:=0; a2:=1; a3:=1;
while a3<n do
    begin a1:=a2; a2:=a3; inc(a3,a1); end;
if n=a3 then begin writeln('You can win!'); halt; end; {So la thua chac}
if ((n-a2)*2>=a2) or (n-a2>Youget*2)
    then Work(n-a2)
    else begin
        writeln('I: ',n-a2);
        dec(total,n-a2); writeln('Remain: ',total);
    end;
GetYourAnswer(n-a2); {Doc so la nguoi choi lay lan nay}
Work(a2-Youget);
end;

begin {Chuong trinh chinh}
write('Total card: '); readln(total);
Youget:=(total-1) div 2; {Gioi han so la toi da may tinh co the lay}
Work(total); {Thiet ke cach lay de may tinh lay duoc la cuoi}
writeln('I win!');
end.

```